Informe técnico · TPI Food Store · Segunda entrega

Estudiante: Santiago Barretto

Repositorio: DB_Santiago_Barretto

Alcance: Unidades 4 a 8 · PostgreSQL 18, MongoDB y SQLite

Rama de esta integración: codex/tpi-segunda-entrega

Elementos implementados y decisiones

Unidad 4. Se verificó ResponsableControlID -> DepositoID, se descompuso control_lote_almacen en dos relaciones FNBC y se creó una vista de compatibilidad. La tabla original se conserva para reversión. Para el reporte de top de categorías se escogió una vista materializada: sólo deriva datos de las tablas normalizadas y se refresca mediante triggers. Se aceptó el costo de refresco en laboratorio; falta medirlo bajo concurrencia. El ensayo de reversión adicional desmontó las tablas derivadas dentro de una transacción y terminó con ROLLBACK, conservando las tres filas del origen.

Unidad 5. Se separaron roles de lectura, escritura, soporte, reportes, auditoría y administración, con permisos por objeto y columna. Se habilitó la auditoría nativa en laboratorio y se simuló un usuario de soporte con autenticaciones fallidas y lectura masiva. Se corrigió el permiso sobre auditoria_evento: admin_datos ahora sólo tiene SELECT. La contención propuesta es preservar el log y suspender el login ante evidencia de compromiso; no se suspendió un usuario de laboratorio legítimo.

Unidad 6. Se prepararon chequeos de salud, ANALYZE condicional, respaldo custom con SHA-256 y manifiesto, y restauración en una base distinta. Se evita VACUUM FULL como rutina y no se almacenan credenciales en Git. En esta integración se registró una tarea diaria de Windows a las 20:00, se comprobó un disparo por horario con resultado 0 y se probó una falla simulada del respaldo. El chequeo y el respaldo son diarios, la revisión de mantenimiento los lunes y la limpieza queda a cargo del autovacuum. El logon Interactive exige sesión de Windows iniciada.

Unidad 7. Se modelaron pedidos con líneas embebidas en MongoDB y se compararon con PostgreSQL. Se mantiene PostgreSQL como origen autorizado; MongoDB puede servir lecturas de pedidos completos, pero no se midió latencia. El origen no tiene pedido.estado, por lo que se documentó explícitamente sin_estado_en_origen en lugar de inventar un estado.

Unidad 8. Se creó una base SQLite STRICT para catálogo, borradores, ítems y cola de sincronización. Un semáforo compara instantáneas con la copia PostgreSQL y clasifica SIMULADO_APROBADO, CONFLICTO o RECHAZADO. No confirma ventas ni escribe en PostgreSQL; un conflicto de precio se resolvió creando una solicitud nueva con otro UUID tras aceptar el precio actualizado. Las specs tienen formato Kiro, pero fueron redactadas con Codex; OpenCode produjo propuestas y una revisión, no el código final.

Cómo se probó y resultados observados

Unidad	Prueba y resultado real
4	DF sin conflictos; EXCEPT bidireccional con 0 diferencias. Reporte: 49,870 ms normalizado y 0,344 ms materializado; auditoría del refresco con 0 diferencias. Reversión ensayada: 3 filas antes/después, vista conservada tras ROLLBACK.
5	Simulacro: f,f,f,t en autenticación, 1000 filas leídas, ESCALAMIENTO_BLOQUEADO. Tras corrección, admin_datos: SELECT=t; INSERT/UPDATE/DELETE=f sobre auditoría.
6	Dump custom 6.420.169 bytes con hash; restauración de 5 categorías, 20.000 clientes, 50.000 productos, 200.000 pedidos y 400.000 detalles; inválidos 0. RPO estimado 64,871 s y RTO 2,664 s. La tarea programada disparó automáticamente en laboratorio y terminó con código 0.
7	20 pedidos y 40 líneas en MongoDB; IDs de pedidos 20/20 iguales al origen, comparaciones de consultas concordantes.
8	C1–C7: borrador offline, conflicto de precio resuelto con nuevo UUID, rechazos por stock y baja, duplicado bloqueado, CHECK y FK exigidos. Hash y conteos de PostgreSQL iguales antes/después del semáforo.

El post incidente de Unidad 6 está en tp8/EVIDENCIA.md y el log de restauración; el de seguridad en tp7/docs/informe_incidente.md. El RPO indicado mide edad del respaldo, no pérdida efectiva de transacciones. Ningún tiempo se extrapola a producción.

IA, revisión y decisión humana

Se usaron Codex para preparación y ejecución y OpenCode Plan para propuestas de Unidades 6–8 y la revisión global final. Prompts y respuestas de cada unidad están en tp7/BITACORA.md, tp8/REVISION_OPENCODE.md, tp9/IA_CONTRASTE.md, tp10/docs/bitacora_ia.md y tpi2/evidencia/opencode_revision_global.txt. Se aceptó separar roles y modelar el pedido embebido; se corrigieron campos del esquema real, verificaciones de inmutabilidad, el permiso de auditoría y un falso éxito potencial del respaldo; se descartó atribuir un escalamiento exitoso sólo a un log y una sospecha de restauración sin fallo reproducido. Se inventariaron los scripts con hash y parser; esto no equivale a revisión semántica individual ni ejecución de todos ellos.

Estado formal: la auditoría detallada de los 14 objetivos está en tpi2/checklist_entrega.md. Quedan sin acreditación literal el uso de Kiro en la migración de Unidad 4, la generación de SQLite mediante OpenCode y la revisión individual de todos los scripts. La rama principal enlaza Unidad 8, mientras esta rama contiene sus artefactos completos.